Experiment No. 10

Experiment Name

Character, Digit, or Face Classification using Convolutional Neural Networks (CNNs)

Aim

To design, train, and evaluate a Convolutional Neural Network (CNN) for character, digit, or face classification using a standard image dataset and analyze its performance using appropriate evaluation metrics.

Purpose

Image classification is one of the most important applications of deep learning in computer vision. Convolutional Neural Networks (CNNs) automatically learn hierarchical image features, eliminating the need for handcrafted feature extraction techniques. CNN-based classifiers have become the foundation of modern applications such as handwritten digit recognition, facial recognition, biometric authentication, medical diagnosis, and intelligent document processing.

In this experiment, students will develop a CNN-based image classification model using datasets such as MNIST (handwritten digit recognition), Fashion-MNIST, EMNIST (character recognition), or CelebA (face recognition). The trained model will be evaluated using standard performance metrics to assess its classification capability.

Prerequisites

- Basic knowledge of Python programming
- Familiarity with Digital Image Processing and Computer Vision concepts
- Understanding of Artificial Neural Networks (ANNs) and Convolutional Neural Networks (CNNs)
- Python libraries: TensorFlow/Keras or PyTorch, OpenCV, NumPy, and Matplotlib
- Jupyter Notebook, Google Colab, or Visual Studio Code

Steps

1. Select a standard dataset such as MNIST, Fashion-MNIST, EMNIST, or CelebA for the classification task.
2. Import the required libraries and load the selected dataset into the development environment.

3. Perform image preprocessing by resizing (if required), normalization, and splitting the dataset into training, validation, and testing sets.
4. Design a Convolutional Neural Network (CNN) consisting of convolutional layers, pooling layers, activation functions, fully connected layers, and an output layer.
5. Configure the model using an appropriate optimizer, loss function, learning rate, batch size, and number of training epochs.
6. Train the CNN model using the training dataset and monitor the learning process through training and validation accuracy/loss curves.
7. Evaluate the trained model on the testing dataset using performance metrics such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.
8. Perform predictions on unseen images and compare the predicted labels with the actual labels.
9. Analyze the model's performance, identify possible misclassifications, and discuss factors affecting classification accuracy.
10. Summarize the results and discuss the practical applications of CNN-based image classification in modern computer vision systems.

Questions

1. What is image classification? How does it differ from object detection and image segmentation?
2. Explain the architecture and working principle of a Convolutional Neural Network (CNN).
3. What is the role of Convolutional Layers, Pooling Layers, and Fully Connected Layers in a CNN?
4. Why is image normalization performed before training a deep learning model?
5. Explain the purpose of activation functions such as ReLU and Softmax in CNNs.
6. What is a Confusion Matrix? How is it used to evaluate classification performance?
7. Differentiate between Accuracy, Precision, Recall, and F1-Score with suitable examples.
8. Compare traditional feature-based image classification methods (e.g., SIFT/HOG + SVM) with CNN-based classification.
9. Mention five real-world applications of CNN-based image classification.
10. How can techniques such as Data Augmentation, Transfer Learning, and Hyperparameter Tuning improve the performance of deep learning-based image classification models?